HybridGuard: Cross-Layer Consistency as Integrity Evidence for Android Web Environments

Anonymous Author(s)

Abstract

Web services use browser-visible environment reports to recognize clients and inform security decisions, yet reports can be modified without corresponding device changes. In Android hybrid Web applications, Android Native, WebView Host, and App Web provide complementary observations of the same environment. Their semantic relationships offer potential integrity evidence: an individually plausible Web report may be incompatible with its system or container context.

HybridGuard associates observations from the same hybrid Web execution and checks their semantic compatibility, accounting for unavailable evidence and legitimate cross-API differences. We collect these observations and use controlled fingerprint manipulations and separate fixed-check comparisons to examine the effectiveness and limits of cross-layer references.

Across 14 controlled manipulation configurations, HybridGuard's App detector detects 105/126 effective modifications (83.3%), with no alarms on 252 controlled normal records. A Native memory reference detects 18/18 modifications versus 6/18 for a global Web-only threshold, both with 0/48 normal alarms, showing the additional information supplied by device-specific context. A Native timezone reference and an absolute Web condition each detect 6/6 modifications, with 0/30 and 6/30 normal alarms, respectively; those six alarms arise from legitimate system timezone changes. In a separate WebView Host geometry study, a Host reference and a fixed Web-height threshold each detect 6/6 modifications, with 0/66 and 6/66 normal alarms, respectively; the threshold flags legitimate layout changes. These findings support cross-layer consistency as useful integrity evidence when an observable, semantically related reference remains informative beyond the manipulated report. Neither Native nor Host is a root of trust, and consistency neither proves authenticity nor guarantees detection of coordinated cross-surface manipulation.

CCS Concepts

• **Security and privacy → Web application security**; *Intrusion/anomaly detection and malware mitigation.*

Keywords

Android WebView, environment report integrity, cross-layer consistency, device fingerprint manipulation, semantic references

1 Introduction

Web services use browser-visible environment reports to recognize clients and inform security-related decisions, including authentication [1, 7, 9]. A party controlling the reporting surface can alter these reports without changing the underlying device. Fingerprint spoofing and inconsistent evasive reports are established concerns [8, 11, 12]. Their integrity therefore matters to these security decisions.

In Android hybrid Web applications, Android Native observations describe system and hardware properties; the WebView Host supplies container, configuration, and layout context; and App Web exposes JavaScript-visible reports. These partially overlapping observations concern the same running environment, with complementary meanings [2]. A Web value that appears reasonable in isolation may be incompatible with its system or container context. We ask: *When can cross-layer semantic consistency serve as integrity evidence for Web-visible environment reports?*

Web-reported memory can remain below a global threshold yet exceed what the device plausibly supports; a Native reference supplies device context. A normal system timezone change can preserve compatible observations, whereas changing only the Web report may break their relationship. Normal container-layout changes can affect Web geometry; Host context helps interpret whether a reported size remains compatible with its container. These interpretations depend on measurement semantics, legitimate setting changes, and available evidence.

HybridGuard associates observations from the same hybrid Web execution and evaluates semantic relations with explicit applicability conditions. Controlled fingerprint manipulations provide the experimental setting: they reveal when local report changes leave contradictions and how normal environmental changes affect the same relations.

The evaluated App detector (Full App), which combines within-view and cross-view checks, detects 105/126 effective modifications (83.3%) across 14 controlled configurations, with no alarms on 252 controlled normal records. Separate comparisons of fixed checks explain the value of references: a Native memory reference supplies device-specific context, and the Native timezone reference distinguishes tested system changes from local report modifications. A separate Host geometry study distinguishes tested layout changes from report modifications using a container reference; this check is not integrated into Full App. These comparisons suggest that cross-layer references are useful when they remain observable and semantically informative beyond the locally modified Web report.

Historical normal records expose normal-alarm and evidence-availability costs beyond the controlled benchmark. An optional external Browser extension examines evidence from a separate Web execution on the same device.

Our contributions are:

- A formulation and implemented framework for assessing report integrity through semantic relationships among complementary observations from the same hybrid Web execution.
- Experimental evidence showing how Native references add device context and distinguish local manipulation from normal changes, complemented by a standalone study of Host geometry.

WWW '27 submission draft,
2027.

- A characterization of detection capability, normal alarms, and unavailable evidence that identifies applicability conditions and costs.

2 Cross-Layer Integrity in Hybrid Web Environments

2.1 Multiple Views of One Execution

An Android hybrid Web execution provides three complementary, partially overlapping views. Native supplies system and hardware observations; the WebView Host supplies container, configuration, provider, and layout context; App Web exposes JavaScript-visible reports inside that WebView [2]. Let E denote the relevant runtime state and container context, beyond a static device model or identifier. We write these observations as $N(E)$, $H(E)$, and $W(E)$ without assuming noiseless, complete, or deterministic API outputs. Associating observations from one execution supports comparison, without implying atomic sampling, trusted collection, or hardware attestation.

The implemented App177 measurement contract catalogs 84 Native, 26 Host, and 67 App Web fields. An external Browser runs a separate Web execution on the same device. Browser67 catalogs 67 fields from that execution; paired244 represents linked App–browser observations. These counts describe implementation catalogs, not independent information or the inputs used by every detector.

2.2 Cross-Layer Consistency as Integrity Evidence

Integrity here concerns report manipulation; available evidence limits which relations can be assessed. A local Web-report modification can be illustrated as

$$x_E = (N(E), H(E), W(E)),$$
$$x'_E = (N(E), H(E), W'(E)).$$

Each semantic relation C_j constrains a relevant subset of observations, such as Native–Web or Host–Web. Its interpretation requires an applicability domain, usable operands, and the necessary measurement context. A difference provides evidence only insofar as it violates that measurement constraint.

Local editing may break an applicable relation when other observations still provide distinguishing context: a plausible Web value can then be incompatible with its reference. Some edits remain within compatible ranges. A normal system timezone change may instead move observations together while preserving their relation. Sampling times and implementation differences still limit this interpretation. More views alone do not guarantee a gain; useful evidence depends on informative references and interpretable constraints. This is a mechanism examined empirically, not a necessary-and-sufficient characterization.

HybridGuard evaluates concrete checks using their current required observations. E is explanatory, not an available ground-truth label: the detector neither reconstructs E nor proves that all observations describe a single authentic environment. The notation does not imply access to pre-change baselines, recovery records, or experiment-stage labels. C_j describes compatibility, whereas $r_j(x)$ in Section 3 denotes a detection condition's state.

The three-view model does not require every relation to use all views or imply a triangle of validated checks. Full App combines within-view and cross-view checks; Host geometry remains a separate, unintegrated study; no standalone Native–Host detection rule has been validated.

2.3 Adversary and Scope of Integrity Claims

The tested manipulations alter selected resource, browser, locale, display, and graphics reports through controlled tools, browser emulation, or runtime overrides, without necessarily changing the physical device. Only a measured effect at the target surface qualifies as an *effective modification*; operations with no observable effect remain separate.

Native and Host can also be manipulated. A reference left unchanged by a tested intervention is neither a trusted root nor evidence of statistical or administrative independence. Coordinated changes across compared views, collector control, or backend compromise may evade checks; resistance to these capabilities is unproven. Such evasion differs from misses caused by limited implemented relations or observation coverage.

An alarm means an applicable integrity-check condition triggered; it does not confirm fraud, account takeover, malicious intent, or attacker identity. No alert does not establish authenticity.

2.4 Normal Variation and Missing Evidence

Normal system timezone, language, orientation, and application-layout changes affect comparability, as do API precision, units, support, and container state. Semantic relations can account for unit conversion, coarse reporting, measurement intervals, and container applicability. Legitimate differences require interpretation within these constraints; they are not automatically violations or unknown outcomes.

Observation availability, a relation among observations, and the detector's final output are distinct. For a selected condition, T marks a triggered alarm condition; F means it does not trigger, without certifying authenticity; U means required evidence is unavailable or insufficient, or the relation is inapplicable. Execution or association failure is recorded separately as FAILED. An empty model also has a separate status, not an ordinary normal result.

A missing required operand can prevent judging its relation. An unused field's absence does not independently fail the detector, and a check using only some views does not require all three to be complete. Missing evidence is not filled in as normal. Section 3 explains how HybridGuard implements these conditional relations and evidence states as executable checks and detectors.

3 HybridGuard

HybridGuard implements the conditional relations of Section 2 as executable checks on associated observations. It preserves evidence states and constructs detectors under normal-alarm and coverage constraints; frozen App and App–browser models then evaluate current inputs. Figure 1 separates collection, semantic checking, and offline construction.

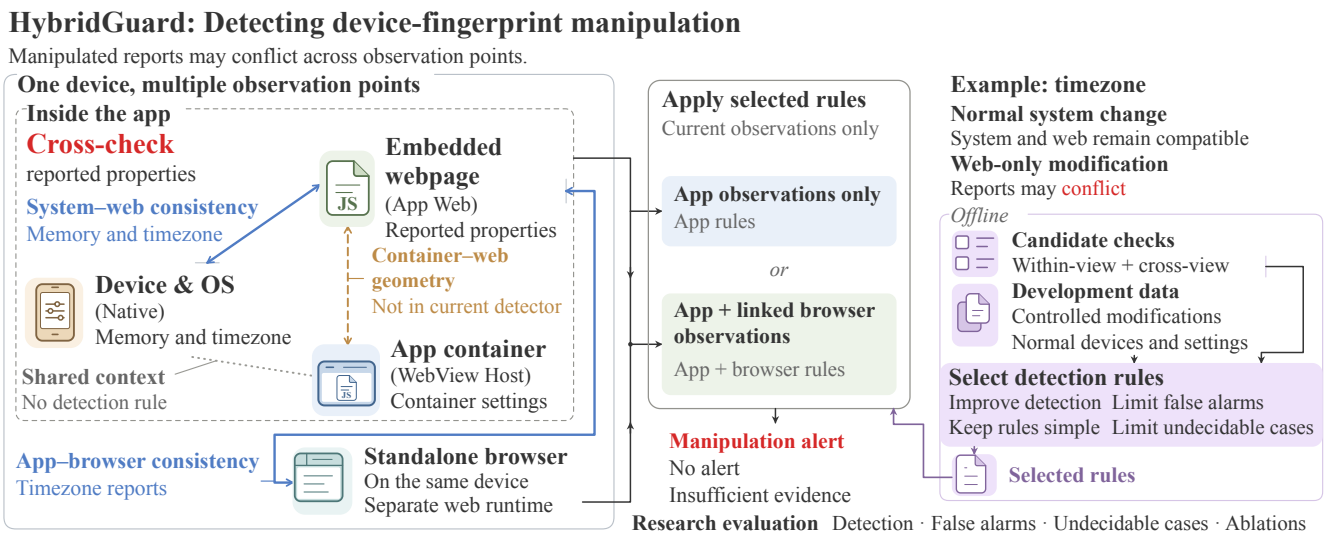


Figure 1: HybridGuard connects current observations to semantic checks. Offline construction constrains normal alarms and definite-output coverage; frozen models support App-only and linked App–browser evaluation. Solid comparison edges identify evaluated relations. Host–Web geometry is a separate study; the dotted Native–Host edge supplies context only. No surface is trusted ground truth. No alert does not establish safety, and execution failures are recorded separately.

3.1 Provenance-Bound Observation Collection

Comparison requires associating values with the execution and collection context they describe. Under the contracts in Section 2, Native, Host, and App Web observations retain values and field states. Original records, receipts, and association metadata support source tracing and validity checks.

Current collection intervals and stability information support applicable checks. In the geometry study, Host-before/after observations surround the Web snapshot within one collection; they are not attack-before and recovery-after records. Operation labels, experimental phases, and other pre-intervention or restoration records do not enter a current prediction.

Valid linkage to the external Browser forms paired observations; incomplete collection preserves App-only data without inventing a Browser vector.

3.2 State-Aware Semantic Consistency

An executable check combines operands, source and validity requirements, measurement interpretation, applicability conditions, and a trigger. Units, coarse reporting, and time context are handled according to the relevant API semantics. For a relation-conflict check, usable and applicable evidence violating C_j triggers $r_j(x)$. The pool also contains single-field conditions, within-view relations, and admitted polarities; these are not all negations of C_j.

For current observation x, $r_j(x) \in \{\mathsf{T}, \mathsf{F}, \mathsf{U}, \mathsf{FAILED}\}$. An empty selected set has its own status. Otherwise, the detector validates inputs required by selected checks: relevant execution or association failures yield FAILED before considering triggers. Without such failure, any T yields an alert; a nonempty set of all F yields definite no-alert; remaining unresolved cases yield U. Thus T can coexist with another condition's U, but cannot override FAILED. Ordinary missing or inapplicable operands remain U, not F; unselected-field problems do not independently fail the detector.

3.3 Cross-Layer Integrity Relations

Native–Web memory. Native memory supplies device context through an upper envelope on the Web report. Both operands must be observed, finite, positive GiB values from the same bound App record, with $m_{\text{native}} \geq 0.25$. On these valid, applicable inputs, the trigger is

$$r_{\text{mem}}(x) = \mathsf{T} \iff m_{\text{web}} > 2^{\lceil \log_2 m_{\text{native}} \rceil}. \tag{1}$$

The power-of-two envelope accommodates coarse reporting and permits lower Web exposures. It detects upward violations of this envelope; downward, within-envelope, or coordinated reports may remain compatible. Unusable measured operands yield U; binding failures retain FAILED. The fixed Web-only comparator is $m_{\text{web}} > 8$ GiB.

Native–Web timezone. The Native IANA zone supplies context for interpreting a Web offset. For time-dependent zones, the current device collection interval and fixed timezone database 2026c determine the effective UTC offset. The check compares its negative with Web `getTimezoneOffset`, using UTC-minus-local minutes. Supported fixed-offset IDs can be evaluated without a date when consistent with the observed Native standard offset. The comparison concerns effective offsets, allowing different names with the same offset.

Missing reliable intervals for time-dependent zones, ambiguous transition intervals, or unusable measurement evidence yield U; zero and minus one are valid values. Database or association failures follow the FAILED policy. A normal system-zone change can remain compatible when the measured offsets move together, while

a local Web override can violate the relation. Collection timing and applicability still bound this interpretation.

Host–Web geometry. The containing View supplies geometric context outside the webpage's own measurements. For either axis, P is the CSS visual-viewport size multiplied by device-pixel ratio and viewport scale; the local scalar H_{view} is the Host View size minus its own padding. The check triggers if either axis satisfies

$$P > H_{\text{view}} + \epsilon,$$
$$\epsilon = 2 + 4 \cdot 2^{-23} \max(|P|, |H_{\text{view}}|, 1) \quad \text{pixels.}$$

Snapshot binding and Host stability before and after that Web snapshot are prerequisites. Stability is assessed independently of Host–Web agreement, preserving stable contradictions as evidence. Valid dimensions, pixel ratio, and scale must be finite and positive. The implemented domain requires an attached, visible View, zero padding, and an identity View transform; outside-domain measurements yield U, while binding failures yield FAILED. The tolerance is an engineering choice, not a platform guarantee. This specialist check remains separate from Full App and its candidate pool.

3.4 Constrained Detector Construction

Checks need detection information, normal-data availability, and bounded alarm and inconclusive-output costs. Maximizing controlled detections alone does not enforce these requirements. The App pool comprises 50 base templates and three fixed relations: Native memory, Native timezone, and same-Web viewport. Training-side numeric encoding and threshold expansion produce 65/65/56 atoms before admission, which considers admissible polarities. Historical evaluation records do not supply thresholds.

For selected singleton-OR clauses S, the sparse stage uses

$$J(S) = \frac{1}{K} \sum_{k=1}^{K} \overline{d}_k(S) - 0.005\, C(S), \tag{2}$$

where $K = 14$. Each $\overline{d}_k$ averages whole-detector detections over effective positions within each training environment, then equally across environments. Complexity $C(S)$ counts clauses plus literal occurrences, distinct from compatibility relations C_j. Greedy improving additions, the best feasible prefix, and backward pruning constitute a heuristic without global-optimality guarantees.

Constraints allow at most eight singleton OR clauses, complexity 16, two conditions per registered family, and no opposite polarities of one atom. Controlled and historical training-normal alarm budgets are 8/168 and 31/630. Historical-normal candidate assessability and final definite-output coverage must each reach 90%; controlled coverage must reach 80% per phase. Fits disallow execution failures and have a 60-second deadline. Appendix A details training-side admission.

Retention starts from each configuration's own sparse solution and preserves existing training detections. Additions increase semantic coverage; same-group swaps use registered higher-quality alternatives and additionally forbid new controlled-normal alarms or losses of definite controlled outputs. All proposals remain feasible. Retention does not continue optimizing Equation 2. Evaluation uses fixed saved retention models, without selecting stages or configurations per result cell.

3.5 Additional Observation Surface: External Browser

The extension tests whether a separate Web execution adds evidence beyond App observations. It is an optional evaluation mode, not a sequential fallback. The retained cross-endpoint extension, Full App +C1, combines Full App with condition C1, which triggers on unequal, validly linked App Web and Browser offsets using the same Web sign convention.

Both operands require observed status, observed-value quality, and finite numeric, non-Boolean values; zero and minus one are valid. Unavailable or inapplicable operands yield U; association failures retain FAILED precedence. C1 has no timestamp gate and does not inherit Native timezone interval handling, leaving asynchronous normal setting changes unresolved.

Construction enumerates no addition, C1, language condition C2, and C1+C2 on the frozen App base. Normal-alarm budgets are 31/630, 0/12, and 1/34 for historical training, pilot, and matched normals, respectively, with 90% definite outputs per normal group and 90% historical candidate assessability. C2 alarms on two matched normal preference changes; C1+C2 also has only 566/630 definite historical training outputs. Neither satisfies the constraints. This selects an increment without retraining a joint classifier over all fields. Original App-benchmark positions lack contemporaneous Browser counterparts, so paired evaluation remains separate.

4 Evaluation

4.1 Research Questions and Materials

We examine detection on controlled App modifications (RQ1), the contribution of semantic references through fixed-check studies (RQ2), normal-alarm and evidence-availability costs using historical normals (RQ3), and incremental evidence from paired App–browser observations (RQ4). Together, these comparisons examine when consistency supplies useful integrity evidence.

Controlled App benchmark. The 378 record positions cover 14 manipulation configurations, three repetitions, and three Android emulator environments (API 29, 30, and 36 with SwiftShader). Each admitted run contributes a normal pre-record, an effective modification, and a confirmed normal post-record: 126 modifications and 252 normals. Post-records serve evaluation and control, not earlier predictions. Three leave-one-environment-out configurations each use 252 training records (84 modified, 168 normal) and 126 held-out records (42 modified, 84 normal). Disjoint held-out records are aggregated once. Development use of these environments and recipes precludes interpreting this split as unseen-tool or independent physical-device attack generalization.

Historical normal records. The mobile test-cloud (MTC) material comprises 891 representative records: 630 discovery records for constraints and tree training, 144 historical development records, and 117 historically reserved evaluation records. Representatives are manufacturer/model/Android-release combinations, not independent physical devices. The latter groups remain separately reported. Historical evaluation records were also used in development. Normal means absence of the study-defined target manipulation, without certifying authenticity or absence of automation and debugging software.

Specialist and paired material. Memory contributes 18 effective modifications and 48 normals, with six additional operations having no observable effect. Timezone contributes six modifications and 30 normals, including six normal system changes. Geometry contributes six modifications and 66 normals, including six normal layout changes. The system and layout changes remain subsets of their normal denominators. App–browser material comprises 60 paired positions, with 14 modifications and 46 normals: 18 pilot and 42 matched positions, including 12 and 34 normals, respectively. These materials have distinct roles and development histories; their denominators are not pooled into an accuracy or independent-device population.

Methods and measures. Full App combines within-view and cross-view checks. Four ablations rebuild their own sparse and retention stages: no MTC alarm cap, Web-only, no memory relation, and no timezone relation. Removing the cap relaxes only that alarm budget, retaining MTC data, assessability, coverage, and other normal constraints. Web-only changes both information and constructible relations, precluding a pure single-factor causal interpretation. Fixed specialist checks and Full App +C1 retain separate identities.

The App tree uses the same fold-specific admitted candidate-state representation, with T/F/U indicators, maximum depth three, minimum leaf size two, and Gini splitting. Training weights allocate one half to modifications, one quarter to controlled normals, and one quarter to MTC normals; modification weight is balanced across configurations. Appendix A gives further settings. This comparison fixes representation and capacity, without testing the strongest classifier over raw fields. Rules use saved `RETENTION` outputs; the tree uses saved `FITTED` outputs.

A detection is an alarm on an effective modification. Normal alarms are reported separately. Separate counts retain `F` for no alert, `U` for unknown, `FAILED` for failure, and `EMPTY_MODEL` for an empty detector. Unknowns remain in their denominators and count as neither normal decisions nor successful rejections. Outputs from multiple configurations do not add record positions or constitute independent replications; equal aggregate counts need not imply identical record-level outputs. Comparisons are descriptive; we report no significance tests, confidence intervals, or population estimates.

4.2 RQ1: Can Cross-Layer Consistency Expose Manipulated Web Reports?

Full App detects 105/126 effective modifications (83.3%), with no alarms on 252 controlled normal records. This establishes detection capability on the benchmark, although its mixture of within-view and cross-view checks prevents attributing all 105 detections to cross-layer relations.

Table 1 compares the alternatives. Web-only and the ablation without memory each detect 96/126, losing nine resource detections. Removing the timezone relation leaves 105/126 because an absolute-offset condition substitutes on these materials; normal system changes in Section 4.3 test the apparent redundancy. Removing the MTC cap raises detection to 108/126 through three additional screen detections, whose normal cost is examined in Section 4.4. The fixed App tree detects 72/126, characterizing this representation and capacity rather than a general advantage of rules over machine learning.

All methods miss nine webdriver-only and nine WebGL-identifier modifications. Historical raw WebGL unavailability and loss of original webdriver-false semantics prevent corresponding candidates from meeting assessability requirements. These misses identify limits of the admitted representation, without establishing intrinsic undetectability.

The table groups 14 configurations into five groups using their complete state-count vectors across all six methods. Each original configuration retains nine modifications, with group sizes preserving weight. Aggregate detection establishes capability; the next comparisons explain what references contribute.

4.3 RQ2: Why Do Cross-Layer References Help?

Table 2 compares fixed checks, distinct from complete-detector ablations. The three cohorts retain separate denominators and test complementary roles for references: additional device context, interpretation of legitimate co-change, and container context.

Memory: additional device context. The Web-only > 8 GiB condition detects 6/18 modifications, while the Native memory reference detects 18/18; both have 0/48 normal alarms. The additional 12 detections are upward changes to 4 and 8 GiB. Both detect the six changes to 16 GiB. Six operations targeting 2 GiB have no observable effect and trigger neither check; they belong to neither the effective-modification denominator nor the 48 normals. Native memory supplies device-specific context absent from the global threshold. This supports the tested upward comparisons, without evaluating downward, within-envelope, or coordinated modifications. These 12 additional specialist detections are separate from the nine resource detections lost in the App benchmark.

Timezone: interpreting legitimate co-change. Native-reference and absolute-offset checks both detect 6/6 modifications, but produce 0/30 and 6/30 normal alarms, respectively. All six alarms arise from normal system timezone changes. The reference interprets the tested offsets moving together, whereas the absolute condition alarms on that movement. Thus, removing the relation without changing the main benchmark's detection total does not establish its redundancy. Separately reported complete-model replay supports this distinction: no-timezone and Web-only also alarm on those six normal changes, while Full App produces no alarms.

Geometry: contextual evidence from the Host. The fixed Web-height threshold detects 6/6 modifications but alarms on 6/66 normals. The same-Web viewport relation detects 0/6 with 0/66 normal alarms. The Host geometry bound detects 6/6 with 0/66 normal alarms, distinguishing the tested changes while retaining detection. The same-Web result shows why low normal alarms alone do not establish informative checking.

The Host check remains an independent specialist and is not integrated into Full App. Full App configurations 01/02 still use the old height condition and alarm on the six normal layout changes; configuration 03 neither detects the six modifications nor alarms on those changes. The Host result cannot replace these complete-model outcomes.

Table 1: App-side detection and component comparisons. Cells give detections out of nine effective modifications per original configuration; parentheses give differences from Full App. Fourteen configurations are grouped only when their complete six-method state-count vectors agree. Adjacent equal results are merged. Group sizes retain the original weighting.

Modification group	Group size	Full App	No MTC alarm cap	App Web only	No memory relation	No timezone relation	App tree
CPU + memory	1	9		0 (−9)		9	0 (−9)
Screen metrics	1	6	9 (+3)	6			0 (−6)
Timezone; plugins/MIME	2	9					0 (−9)
Shared detections†	8	9 (all detected)					
Webdriver; WebGL IDs	2	0 (all missed)					
Total detections	14	105/126	108/126	96/126	96/126	105/126	72/126

Gray, green and red indicate equal, more and fewer detections than Full App, respectively; colors do not rank overall method quality. †Playwright, Puppeteer, CDP multi-field and Stealth presets; platform, UA, UA + platform and language-list modifications. All methods have 0/252 controlled-normal alarms; unknown, failed and empty-model counts are zero. Rules use saved RETENTION results and the tree uses FITTED results. The extra three screen detections without the cap have a separate normal-record cost (Table 3).

Table 2: Specialist comparisons of fixed checks. Device-reference conditions (*) supplement detection or tolerate normal changes in these separate cohorts. Entries are detection and normal-alarm counts; the denominators appear once per group. These are individual checks, not complete-model ablations.

Specialist cohort	Fixed condition	Detected	Normal alarms
Memory 18 modifications; 48 normal	Web memory > 8 GiB	6	0
	System-memory upper bound*	18	0
Timezone 6 modifications; 30 normal	Absolute timezone offset	6	6
	System-timezone reference*	6	0
Screen 6 modifications; 66 normal	Fixed Web-height threshold	6	6
	Same-Web viewport relation	0	0
	Host-geometry upper bound*	6	0

Six memory operations with no observable effect produce no alarms and are excluded from the 18 effective modifications. Normal timezone changes (6) and layout changes (6) are subsets of their 30 and 66 normal records. Unknown and failed counts are zero. The host-geometry check remains a standalone specialist condition; it is not integrated into Full App.

4.4 RQ3: What Are the Normal-Alarm and Availability Costs?

The normal-record comparisons examine two costs: alarms on legitimate observations and loss of definite decisions. Table 3 places the gain from 105/126 to 108/126 detections against alarms on all 144 historical development and all 117 historical reserved evaluation records after removing the cap. With the cap, configurations 01/02 each produce one alarm in each group; configuration 03 produces none. Both methods still have 0/252 controlled-normal alarms. Those controlled normals therefore fail to expose the much larger historical-normal cost.

Saved cap-free models select broad conditions, including hardware concurrency > 1 and language-list length > 1. They satisfy the controlled-normal budget while covering many MTC normals. This reveals insufficient coverage of normal conditions by the controlled set, beyond a modest threshold tradeoff. On the separate 630 training normals, capped configurations produce 6, 6, and 0 alarms, respectively, with 51 unknowns each; uncapped configurations alarm on all 630. These training outcomes are not added to either evaluation denominator, nor are model outputs pooled as extra records.

Evidence availability imposes another cost. Each FULL APP configuration has 8/144 and 6/117 unknowns, or 14/261 within that configuration. Removing the memory reference reduces this to 1/261 but loses resource detections. Binary tree outputs do not recover missing measurements or establish more complete evidence. Low alarms do not imply high definite-output coverage. Detection gains must therefore be read with normal alarms and evidence availability.

4.5 RQ4: Does an Additional Observation Surface Add Integrity Evidence?

Table 4 tests the increment from the retained C1 condition on 60 paired positions. For App timezone, App language, Browser timezone, and Browser language, FULL APP detects 2/2, 0/2, 0/5, and 0/5, respectively. FULL APP +C1 gives 2/2, 0/2, 5/5, and 0/5: totals rise from 2/14 to 7/14. All five additional detections concern Browser timezone; neither language family gains detection.

Both methods have 0/46 normal alarms. Counts agree across three frozen configurations and are displayed once. Historical normals reveal an availability cost with unchanged alarm counts: unknowns rise from 8/144 to 11/144 in historical development and

Table 3: Normal-record outcomes with and without the MTC alarm cap. Removing the cap makes every record in both historical evaluation cohorts trigger an alarm, while controlled detections increase only from 105/126 to 108/126. Both methods still have 0/252 controlled-normal alarms. Counts are records, with unknown outcomes reported separately.

Historical cohort	Model configuration	With cap (Full App)			Without cap		
		Alarm	No alarm	Unknown	Alarm	No alarm	Unknown
Evaluation ($N = 144$)	01/02	**1**	135	8	**144**	0	0
	03	**0**	136	8			
Reserved ($N = 117$)	01/02	**1**	110	6	**117**	0	0
	03	**0**	111	6			

Equal counts are merged for display; each configuration uses the same records, so denominators are not pooled. Failed and empty-model counts are zero. The separate 630 constraint-training records yield 6/6/0 alarms and 51 unknowns per capped configuration (01/02/03), versus 630 alarms and no unknowns without the cap. These historical cohorts are not a new blind evaluation.

Table 4: Increment from the retained cross-endpoint timezone check C1. All five additional detections are Browser timezone modifications; App timezone detection is preserved and both language families remain undetected. Bold marks changed counts, not statistical significance.

Method	App modifications		Browser modifications		Total detected	Normal alarms
	Timezone	Language	Timezone	Language		
Full App	2/2	0/2	0/5	0/5	2/14	0/46
Full App + C1	2/2	0/2	**5/5**	0/5	**7/14**	0/46
Count difference	0	0	**+5**	0	**+5**	0

These 60 paired positions have development-use history (14 effective modifications, 46 normal); unknown and failed counts are zero. Counts agree across three frozen configurations and are shown once, without pooling or claiming equal record-level outputs. On the separate historical MTC cohorts (144 and 117 records), alarms are unchanged within each configuration while combined unknowns increase from 14 to 17 out of 261.

remain 6/117 in historical reserved evaluation. Within each configuration, the combined count thus rises from 14/261 to 17/261.

C1 has no timestamp gate, so 0/46 does not establish general robustness to asynchronous normal setting changes. The evidence supports a targeted increment from another observation surface, without establishing complete cross-endpoint detection or an automatic benefit from more views.

4.6 Cross-RQ Findings

The comparisons identify three linked findings. First, references can supply context that isolated Web checks lack: Native memory distinguishes tested reports accepted by a global threshold, and Host geometry supplies container information beyond the same-Web relation. Second, interpreting legitimate co-change can preserve detection while reducing particular normal alarms. The timezone and geometry comparisons show why this value can be hidden by an unchanged modification-detection total.

Third, gains depend on the relation's information, applicability, and available evidence. The same-Web geometry misses, historical alarm costs, and additional C1 unknowns make these dependencies observable. In these comparisons, the useful references provide measured context that distinguishes the reports at issue; adding a view alone does not settle that question. Taken together, these results support a conditional mechanism interpretation for the studied materials, rather than a universal integrity criterion. Section 5 discusses the resulting limits on attribution, generality, and integration.

5 Discussion

5.1 What Counts as Integrity Evidence?

The reference benefits in Section 4.3 concern contextual compatibility under the applicability conditions in Section 2. A violated relation motivates scrutiny of report integrity without identifying which observation was altered or establishing malicious intent. Normal changes, sampling times, and implementation differences require interpretation within those conditions before a discrepancy can support an attack hypothesis. Conversely, modified observations can remain compatible, so an absent trigger does not establish authenticity.

The evidence comes from finite recipes in three emulator environments; repeated runs and development exposure limit independence. MTC normals broaden observed normal conditions without supplying independent physical-device attack generalization. Retrained ablations characterize entire constrained alternatives, not single-factor causal effects for arbitrary devices. The fixed-capacity tree is likewise not a comprehensive comparison with tuned advanced classifiers.

5.2 When Cross-Layer Consistency Fails

Historical field availability, lost webdriver-false semantics, and assessability admission limit the candidates for webdriver and WebGL, revealing representation gaps rather than intrinsic undetectability. The tested language candidate C2 exceeded the normal-alarm budget on legitimate preference changes. These findings limit the

tested candidates, without establishing that language observations lack information.

Other modifications can remain within a checked relation. With valid operands, an altered Web memory value at or below the Native-derived envelope does not trigger that condition. This follows from its predicate; the experiments do not cover all downward or within-envelope modifications.

Unavailable or ambiguous required evidence, or unmet applicability conditions, leave a relation unassessed. The states defined in Sections 2 and 3 distinguish this from an observed no-alert result or an execution failure. Current misses help locate unmet conditions, but establish neither necessary-and-sufficient detection conditions nor an automatic benefit from more observations.

5.3 Adaptive and Coordinated Adversaries

Even with usable evidence, reports preserving the checked relations may be indistinguishable from compatible normal observations under those checks. Native and Host are not trusted roots; different observation positions imply neither statistical nor administrative independence. Evasion depends on the selected checks' actual dependencies, without necessarily requiring control of all three surfaces or all fields. Some local modifications may remain compatible without changes elsewhere.

Adaptive adversarial optimization and comprehensive coordinated attacks have not been evaluated. This threat analysis supplies neither bypass rates nor evidence of increased attacker cost. Consistency cannot substitute for trusted attestation.

5.4 Deployment, Drift, and Missingness

Changes in WebView providers, API exposure, system versions, locale conventions, container behavior, or device coverage may alter measurement semantics and applicability. These are potential drift risks requiring longitudinal and cross-version validation. Observation association is not atomic sampling. C1 has no timestamp gate, and low alarms on the paired normals do not establish general robustness to asynchronous setting changes.

Adding references can increase inconclusive outputs, as C1 does on historical normals. Unknown cannot automatically mean normal, attack, or authentication failure. Deployment would require designed and validated handling of insufficient or conflicting evidence; automatic fallback, calibrated risk, and an authentication decision process are not established by this study.

Integrating the Host geometry specialist would require joint evaluation of detection, normal alarms, availability, and complexity. Real devices, window modes, keyboard interaction, and padding need further validation; the current zero-padding domain remains unchanged. Nor has independent incremental value been demonstrated for every one of the 177 App or 244 paired fields.

5.5 Privacy and Ethics

Collecting environment information for integrity checking can also enable identification, linkage, and tracking, as discussed in WebView research and W3C guidance [13, 14]. Controlled manipulations used emulators; historical normals came from cloud-device collection. The documented MTC collector requested network access and network-state observation, without contacts, SMS, photos, location, camera, or microphone permissions. Limited permissions remove neither fingerprinting privacy risks nor the need to determine applicable review and authorization requirements.

The paper package supplies derived counts, condition definitions, and table sources, without raw device payloads, pair tickets, or stable device identifiers; this scope does not extend to the repository as a whole. Institutional review, exemption, and participant-consent status are not established by the available study documentation.

Future use should minimize retained observations and handle uncertain evidence cautiously. The experiments do not justify automatic denial of service, rejection of identity, or high-stakes security decisions based solely on a fingerprint alert.

6 Related Work

6.1 Browser Fingerprinting and Spoofing

Report manipulation and relationships among fingerprint attributes are established research topics. Gummy Browsers studies targeted impersonation through fingerprint copying and browser modifications [8]. Fp-Scanner examines inconsistencies introduced by countermeasures, including disagreement between JavaScript and HTTP user-agent reports and constraints involving browser, OS, and rendering behavior [11]. Picasso checks claimed browser and OS classes using canvas challenge–response and a server-side response dictionary [3]. Fp-Scanner and Picasso already use inconsistencies or behavioral evidence to scrutinize reported attributes.

Measurement studies provide complementary evidence. Him of Many Faces analyzes benign and adversarial fingerprints collected through website JavaScript and native mobile SDKs, covering network, browser, and user attributes [15]. Its traffic classifications do not directly label whether an environment report was modified. FP-Inconsistent detects evasive bots using spatial incompatibilities within requests and temporal changes across requests associated through cookies or IP addresses [12]. Its analysis already identifies consistency-preserving evasion and normal-user alarm risks.

BrowserFM connects feature models of fingerprints, system hardware/software, and browser configurations to represent dependencies and analyze configuration changes [6]. Structured relations and associated context therefore have clear precedents. HybridGuard focuses on semantic references within an Android hybrid Web execution: Native and Host observations contextualize App Web reports, while controlled modifications and normal changes test when those references distinguish manipulation.

6.2 Cross-Browser and Cross-Context Fingerprinting

Fingerprinting supports identification and security uses, as surveyed by Laperdrix et al. [7]. Alaca and van Oorschot analyze fingerprinting for augmenting authentication, including stability and spoofing resistance [1]. Spooren et al. find similarity and predictability limitations in mobile fingerprints collected for an authentication system [9].

Cao et al. derive OS- and hardware-dependent features through browser APIs and rendering tasks, then select features stable across browser pairs for device identification [4]. Their design accounts for browser capabilities and tests rendering stability under window and zoom changes. Such browser-mediated measurements differ from

our associated Native measurements and Host container observations. Both approaches use relationships across observations, but identifying a device across browsers and assessing a current report against contextual constraints ask different questions. HybridGuard evaluates neither identity uniqueness, cross-browser linkage accuracy, authentication success, nor production fraud detection. Its external Browser extension remains a separate execution context, outside the Native–Host–App Web model.

6.3 Android WebView and Cross-Boundary Analysis

WebView research directly overlaps our observation setting. Tiwari et al.'s WVProfiler instruments Android WebView loading calls to associate applications with exposed user agents, headers, and URLs; its case studies examine fingerprint leakage and Java–JavaScript bridge risks [10]. Tracking Without Borders combines Java and JavaScript analysis with runtime and network observation, examining injected information and relationships between application permissions and WebView access [14]. Cross-Boundary Mobile Tracking jointly observes Java calls and JavaScript execution inside WebViews to trace information diffusion and tracking across their boundary [5]. Thus, observing both application and WebView contexts is already an established approach.

HybridGuard examines the semantic compatibility of associated environment observations under explicit applicability conditions. Memory, timezone, and geometry comparisons investigate additional device context and the interpretation of legitimate system or layout changes; unavailable evidence limits assessment. Host geometry remains a separate study, unintegrated into Full App.

We have not compared these systems directly under a unified protocol and claim no empirical superiority.

7 Conclusion

This study examines how cross-layer semantic consistency can provide integrity evidence for Web-visible environment reports in Android hybrid Web applications. HybridGuard associates complementary Native, WebView Host, and App Web observations from the same execution, implements semantic relations as checks with explicit applicability conditions, and studies them through controlled manipulations and normal changes. The Native memory comparison and standalone Host geometry study show that device and container references supply context absent from isolated Web checks, exposing some otherwise missed modifications. Interpreting legitimate system and layout changes also avoids specific normal alarms in the tested comparisons while retaining detection.

These benefits depend on references that provide distinguishing context, relations with defined semantics and applicability, and available evidence. Detection benefits must be assessed alongside normal alarms and inconclusive outcomes; absence of an alarm does not confirm an unmodified environment. Consistency establishes no trusted root, and coordinated modifications preserving checked relations may evade detection. Under the studied conditions, cross-layer semantic consistency therefore offers useful, conditional integrity evidence for Web-visible environment reports.

References

[1] Furkan Alaca and Paul C. van Oorschot. 2016. Device Fingerprinting for Augmenting Web Authentication: Classification and Analysis of Methods. In *Proceedings of the 32nd Annual Conference on Computer Security Applications*. ACM, 289–301. doi:10.1145/2991079.2991091

[2] Android Developers. 2026. Build Web Apps in WebView. Living documentation; accessed 17 October 2026. https://developer.android.com/develop/ui/views/layout/webapps/webview

[3] Elie Bursztein, Artem Malyshev, Tadek Pietraszek, and Kurt Thomas. 2016. Picasso: Lightweight Device Class Fingerprinting for Web Clients. In *Proceedings of the 6th Workshop on Security and Privacy in Smartphones and Mobile Devices*. ACM, New York, NY, USA, 93–102. doi:10.1145/2994459.2994467

[4] Yinzhi Cao, Song Li, and Erik Wijmans. 2017. (Cross-)Browser Fingerprinting via OS and Hardware Level Features. In *Network and Distributed System Security Symposium*. doi:10.14722/ndss.2017.23152

[5] Sohom Datta, Michalis Diamantaris, Ahsan Zafar, Junhua Su, Anupam Das, Jason Polakis, and Alexandros Kapravelos. 2026. Cross-Boundary Mobile Tracking: Exploring Java-to-JavaScript Information Diffusion in WebViews. In *Network and Distributed System Security Symposium*. doi:10.14722/ndss.2026.230910

[6] Maxime Huyghe, Clément Quinton, and Walter Rudametkin. 2025. BrowserFM: A Feature Model-Based Approach to Browser Fingerprint Analysis. In *NDSS Workshop on Measurements, Attacks, and Defenses for the Web*. doi:10.14722/madweb.2025.23017

[7] Pierre Laperdrix, Nataliia Bielova, Benoit Baudry, and Gildas Avoine. 2020. Browser Fingerprinting: A Survey. *ACM Transactions on the Web* 14, 2, Article 8 (2020), 33 pages. doi:10.1145/3386040

[8] Zengrui Liu, Prakash Shrestha, and Nitesh Saxena. 2022. Gummy Browsers: Targeted Browser Spoofing Against State-of-the-Art Fingerprinting Techniques. In *Applied Cryptography and Network Security (Lecture Notes in Computer Science, Vol. 13269)*. Springer, 147–169. doi:10.1007/978-3-031-09234-3_8

[9] Jan Spooren, Davy Preuveneers, and Wouter Joosen. 2015. Mobile Device Fingerprinting Considered Harmful for Risk-Based Authentication. In *Proceedings of the Eighth European Workshop on System Security*. ACM, Article 6, 6 pages. doi:10.1145/2751323.2751329

[10] Abhishek Tiwari, Jyoti Prakash, Alimerdan Rahimov, and Christian Hammer. 2022. Our Fingerprints Don't Fade from the Apps We Touch: Fingerprinting the Android WebView. Preprint. arXiv:2208.01968 [cs.CR] doi:10.48550/arXiv.2208.01968

[11] Antoine Vastel, Pierre Laperdrix, Walter Rudametkin, and Romain Rouvoy. 2018. Fp-Scanner: The Privacy Implications of Browser Fingerprint Inconsistencies. In *27th USENIX Security Symposium*. USENIX Association, 135–150. https://www.usenix.org/conference/usenixsecurity18/presentation/vastel

[12] Hari Venugopalan, Shaoor Munir, Shuaib Ahmed, Tangbaihe Wang, Samuel T. King, and Zubair Shafiq. 2025. FP-Inconsistent: Measurement and Analysis of Fingerprint Inconsistencies in Evasive Bot Traffic. In *Proceedings of the 2025 ACM Internet Measurement Conference*. ACM, 134–149. doi:10.1145/3730567.3732919

[13] W3C Privacy Working Group. 2025. *Mitigating Browser Fingerprinting in Web Specifications*. W3C Group Note. World Wide Web Consortium. Editors: Nick Doty and Tom Ritter. Published 25 September 2025; accessed 17 October 2026. https://www.w3.org/TR/fingerprinting-guidance/

[14] Nipuna Weerasekara, José Miguel Moreno, Srdjan Matic, Joel Reardon, Juan Tapiador, and Narseo Vallina-Rodríguez. 2025. Tracking Without Borders: Studying the Role of WebViews in Bridging Mobile and Web Tracking. *Proceedings on Privacy Enhancing Technologies* 2025, 4 (2025), 745–762. doi:10.56553/popets-2025-0155

[15] Shujiang Wu, Pengfei Sun, Yao Zhao, and Yinzhi Cao. 2023. Him of Many Faces: Characterizing Billion-Scale Adversarial and Benign Browser Fingerprints on Commercial Websites. In *Network and Distributed System Security Symposium*. doi:10.14722/ndss.2023.24394

A Configuration and Artifact Details

This appendix documents display compression and reproducibility details. The main text contains all results needed for its conclusions. The companion source package includes the four English table sources, their saved aggregate data, a source map, and a count-verification script. The script checks saved counts and compression; it does not rerun the detector or establish new experimental validity.

A.1 The Fourteen Configurations

Table 1 preserves five groups. The single-configuration groups are CPU plus memory and screen metrics. The two-configuration group with additional tree misses consists of timezone-only and plugin/MIME modifications.

The eight configurations detected by all methods are four legacy tool presets (Playwright, Puppeteer, CDP multi-field, and Stealth), platform-only, user-agent-only, user agent plus desktop platform, and the language-list preset. The common-miss group contains webdriver-only and WebGL-identifier modifications. A language-list modification here is not interchangeable with the App/browser language operations in Table 4. The companion aggregate-data mapping preserves the exact original recipe identifiers, suffixes, and source rows.

Group sizes are 1, 1, 2, 8, and 2. Multiplying each row's per-configuration detection count by its group size and summing gives 105, 108, 96, 96, 105, and 72 detections for the six methods. Each original configuration contributes nine effective modifications and 18 normal records. The compression tests the full count vector (N, T, F, U) together with failure and empty-model counts, not only the detection count. It preserves a mapping to every original configuration.

A.2 Frozen Models and Comparison Roles

The three App configurations hold out the API 29, API 30, and API 36 emulator environments, respectively. `APP_FULL` denotes the retained `B_REL_TZ` models; `A_NO_MTC_CAP`, `A_APP_WEB_ONLY`, `A_NO_MEMORY_REL`, and `A_NO_TIMEZONE_REL` denote retrained alternatives. `APP_TREE` is the candidate-state tree, and `PAIRED_BASE` is the frozen App detector plus C1. No cell substitutes a specialist condition or another model's best result.

The tree implementation uses scikit-learn 1.7.2, `splitter=best`, `random_state=20261006`, no class-weight override, and a positive-class probability threshold of 0.5. Its fold-specific inputs contain 63, 63, and 54 admitted atoms, each with three state indicators. These counts differ from the full expanded pool because training-side admission is applied. Fifteen positions per controlled held-out set and 73 of the 891 MTC records contain an unknown candidate; the saved decision paths do not use those missing atoms. A binary output is therefore not proof of complete observation.

A.3 Conditions, Unknowns, and Study Boundaries

The memory comparison uses the Native envelope and the fixed Web-memory threshold at 8 GiB. The absolute timezone comparator alerts above −480 minutes; the Native reference uses effective IANA offsets with fixed timezone data. The old screen comparator uses Web inner height greater than 710 CSS pixels. The host condition applies snapshot stability and the tolerance in Section 3; its measured result does not imply integration into the App model.

C1 requires both its field-status and field-quality markers to indicate an observed value. It accepts finite numeric offsets from the linked App Web and browser records, including zero and minus one, and alerts on inequality. Both offsets use the same Web sign convention. It has no timestamp acceptance gate. Pair integrity errors yield failure; absent, nonnumeric, nonfinite, or nonobserved operands yield unknown. The model's selected-input failure precedence and OR semantics still apply.

Across the two historical evaluation groups, Full App's per-configuration totals are $(T, F, U) = (2, 245, 14)$ for configurations 01/02 and $(0, 247, 14)$ for 03. With C1, they become $(2, 242, 17)$ and $(0, 244, 17)$, respectively. These are sums of the 144- and 117-record groups within one model, not additional records. The six no-effect memory operations remain outside effective-modification counts. Newer geometry engineering-smoke positions and later resource-extension experiments are outside the four selected materials and are not pooled into the manuscript results.

A.4 Training-Side Admission and Extension Selection

Numeric threshold expansion uses training-only quantiles at 0.25, 0.5, and 0.75. Candidate support requires at least three complete true/false triplets and at least two triggered modification triplets in at least one bundle/environment, in addition to the normal assessability criteria. These thresholds and the normal budgets are fixed before scoring; historical evaluation records do not supply threshold quantiles.

The browser-extension comparison enumerates four subsets: no addition, C1, C2, and C1 plus C2. Feasible sets are ranked by the equal mean of detection ratios for App timezone, App language, Browser timezone, and Browser language modifications, then by fewer additions and a fixed set order. This is a constrained choice on development-used material. The accepted C1 model is not assembled by selecting a different condition for each evaluation family.